

LangChain Basic

LangChain 기초

02 LangChain 구성 요소

목차 02 LangChain 구성 요소

1. PromptTemplate
2. LLM
3. Chain
4. OutputParser
5. Agent, Tool
6. Memory
7. Callback

1. PromptTemplate

총 8개 핵심 컴포넌트

구성 요소	역할
PromptTemplate	프롬프트 포맷 구성 및 템플릿화
LLM	언어모델 호출
Chain	구성요소들을 순차적으로 연결
OutputParser	생성한 텍스트 응답을 원하는 자료형으로 가공/추출
Agent	목적에 맞는 도구 선택 및 실행
Tool	외부 기능 실행 API, 계산기 등
Memory	대화 상태 유지 및 활용
Callback	로그, 실행 추적 등 후처리



일관된 형태의 프롬프트를 자동으로 생성

- LangChain에서 가장 기본이면서도 중요한 구성 요소
- LLM의 성능은 프롬프트에 크게 의존하기 때문에,
이를 체계적이고 재사용 가능한 방식으로 관리해야 함
- 사용자의 입력값을 미리 정의된 문장 구조에 삽입하여,
일관된 형태의 프롬프트를 자동으로 생성
- 예시
 - `{city}`는 변수 슬롯이며, `.format()` 메서드를 통해 값이 삽입됨
 - 프롬프트가 코드처럼 관리 가능해지고,
파이프라인에서 동적으로 연결될 수 있음

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "{city}의 오늘 날씨를 요약해줘.")
formatted = template.format(city="서울")

# formatted 결과:
# "서울의 오늘 날씨를 요약해줘."
```



LLM의 입력을 체계적으로 관리하고 자동화하기 위해

A

자동화된 생성

프롬프트를 매번
수동으로 작성해야 함



템플릿화하여 반복 작업 없이
자동 생성 가능

B

형식 일관성 유지

프롬프트 일관성 부족



하나의 양식을 기반으로
항상 동일한 구조 보장

C

입력 유연성 확보

동적 입력을 반영하기 어려움



변수 삽입("{}")으로
사용자 요청에 따라
프롬프트 유동 생성

자주 쓰이는 핵심 메서드와 속성

메서드/속성	설명	사용 예시
from_template(...)	문자열 기반 템플릿 생성	PromptTemplate.from_template("Hello, {name}!")
format(**kwargs)	템플릿 변수에 실제 값을 채워 넣음	template.format(name="엘리스")
input_variables	템플릿에서 요구하는 변수 목록 확인 가능	template.input_variables

2. LLM

다양한 언어모델 연동

- LangChain은 다양한 LLM을 손쉽게 사용할 수 있음
- 특정 모델이나 API에 종속되지 않고, 사용 목적이나 과금 정책에 따라 자유롭게 선택하고 교체할 수 있음

```
from langchain_openai import OpenAI

llm = OpenAI(model="gpt-4o-mini")
llm.invoke("서울의 인구는?")
```



제공자	대표 모델	특징
OpenAI	GPT-3.5, GPT-4	대화형 응답 성능 우수, 안정성 높음
Anthropic	Claude 1/2/3	윤리성 중심 설계, 긴 컨텍스트 지원
Hugging Face	FLAN-T5, BLOOM 등 Transformer 계열	오픈소스 모델 활용, 커스터마이징 가능
Cohere	Command R	빠른 속도와 API 기반 분류/요약 특화



LLM을 다양한 상황에 따라 교체하거나 복수의 모델을 동시에 사용하는 설계도 가능

A

고품질 대화형 챗봇

GPT-4 Turbo (OpenAI)



반응성 빠르고 긴 컨텍스트 지원
채팅 응답 품질 탁월

B

저비용 고속 응답

Gemini 1.5 Flash
(Google DeepMind)



속도 빠르고 저렴
실시간 요약이나
분류 작업에 최적

C

문서 요약 및 RAG

Claude 3 Sonnet +
LangChain + Chroma



문서 요약 정확도 높고
벡터 검색과의 궁합이 좋음

D

한국어 특화

HyperCLOVA X (NAVER)



한국어 최적화
한글 QA 및 문서 처리
정확도 높음

3. Chain

모듈을 잇는 LangChain의 중심 구조

- LangChain에서 다양한 구성 요소
(PromptTemplate, LLM, OutputParser 등)를
순차적으로 연결하여 하나의 워크플로우로 만드는 구조
- 단일 구성요소로 처리할 수 없는 **복합적 작업을**
단계별로 분해하고 조립할 수 있음
- 예시
 - 기본 구성 : PromptTemplate → LLM

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import OpenAI

prompt = PromptTemplate (
    input_variables=["city"],
    template="{city}의 오늘 날씨를 요약해줘."
)

llm = OpenAI(model="gpt-4o-mini")

chain = prompt | llm
chain.invoke("서울")
```



구성 요소 간 연결을 통해 자동화·재사용 가능

A

전후 처리 자동화

LLM 호출 전 입력 가공,
후처리 과정 필요



구성 요소를 순서대로 연결해
파이프라인처럼 처리 가능

B

구조화된 재사용성

작업 로직을 코드로 직접 구성하면
반복/비효율 발생



체인으로
재사용 가능한 템플릿 구조 제공

C

복합 작업 구성 유연성

멀티스텝 논리 흐름 구현이
복잡함



다양한 체인 유형으로
단계별 분기 및 흐름 제어 가능



체인에서 순서가 중요한 이유

데이터 흐름의 논리적 순서

올바른 순서

```
# 입력 → 프롬프트 → 모델  
chain = prompt | llm
```

- **prompt** : 사용자 입력을 프롬프트 문자열로 가공
- **llm** : 프롬프트를 받아 응답 생성

잘못된 순서

```
# 입력 → 모델 → 프롬프트  
chain = llm | prompt
```

- **llm** : 프롬프트를 받아야 함 → 처음부터 실행되면 형식이 안 맞음
- **prompt** : 실행 결과가 아닌 **실행 준비 단계**에 해당하는 객체
→ LLM보다 뒤에 위치하면 작동하지 않음

4. OutputParser



LLM 응답의 구조화 처리

- LLM이 생성한 텍스트 응답을 원하는 **자료형으로 가공/추출**하는 모듈
(숫자, 리스트, JSON 객체 등)
- LLM의 출력은 기본적으로 **자연어(비정형 텍스트)**
 - 그대로는 다음 모듈로 전달하기 어려우므로 파싱 필요
 - 특히 체인이나 도구 호출 시 정형화된 결과가 중요
- 예시
 - StrOutputParser : LLM의 응답을 그대로 문자열로 반환
 - chain : "LLM → 파서" 구조의 간단한 체인
 - 입력을 받고 응답을 문자열로 반환

```
from langchain_openai import OpenAI
from langchain_core.output_parsers import StrOutputParser

llm = OpenAI(model="gpt-4o-mini")
chain = llm | StrOutputParser()
chain.invoke("서울의 인구는?")
```

원하는 데이터 형식에 따라 다양한 파서를 선택하여 적용

기능	설명 예시	반환 예시
StrOutputParser()	응답을 그대로 문자열로 반환	"서울의 인구는 약 950만 명입니다."
JsonOutputParser()	응답을 JSON 형식으로 파싱	{"city": "서울", "population": 9500000}
CommaSeparated ListOutputParser()	쉼표나 줄바꿈 기준으로 항목 분리 후 리스트 처리	["사과", "바나나", "포도"]
PydanticOutputParser()	사용자 정의 데이터 모델에 맞게 파싱	{"task": "요약", "length": "3줄"}

연결하는 순서에 주의해야 함

- LangChain의 체인은 함수형 파이프라인처럼 동작
- 입력 → 가공 → 응답 → 후처리의 순서를 반드시 지켜야 함
 - 가공 : 프롬프트
 - 응답 : LLM
 - 후처리 : 파서
- PromptTemplate과 StrOutputParser는
앞뒤 위치가 정해져 있는 모듈

```
from langchain_core.prompts import PromptTemplate
from langchain_openai import OpenAI

prompt = PromptTemplate (
    input_variables=["city"],
    template="{city}의 오늘 날씨를 요약해줘."
)

llm = OpenAI(model="gpt-4o-mini")
parser = StrOutputParser()

chain = prompt | llm | parser
chain.invoke("서울")
```

5. Agent, Tool

LLM 기반의 의사결정 엔진

LLM이 상황을 스스로 판단하여 도구를 선택하고 실행하는 구조

- 사전에 고정된 순서를 따르는 Chain과 달리 **유연**하고 **자율적인 작업 수행** 가능
- **LLM 기반 의사결정**
 - Agent는 입력을 분석한 후, 어떤 Tool이 필요한지를 **LLM이 직접 판단**
 - 문장의 의미를 이해해서 필요한 도구를 직접 골라서 사용
- **Tool 선택 및 실행 자동화**
 - Agent는 도구를 하나만 쓰는 것이 아니라, 필요에 따라 **연속적으로 여러 도구를 호출**할 수 있음



예시 시나리오

“서울의 오늘 날씨와 내일 기온 차이를 알려줘”

날씨 관련 질문임을 판단



Google Search API Tool 선택



오늘/내일 기온 정보를 검색



결과를 정리해 자연어 응답 생성

Agent가 호출하는 외부 기능

LLM이 단독으로 할 수 없는 작업을 보완하기 위해
외부 시스템이나 함수로 연결되는 기능 단위

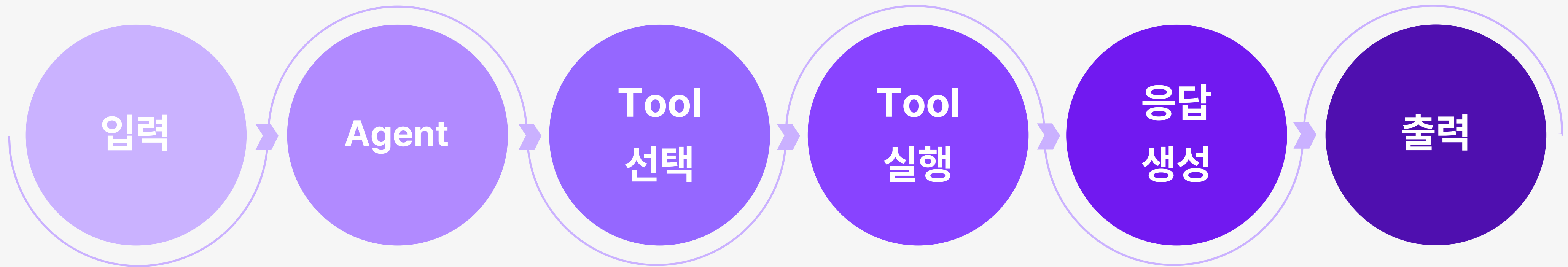
Tool 이름	설명	예시 응답
Python REPL	간단한 수치 계산, 코드 실행	수익률 계산, 날짜 차이, 수학 공식 등
Google Search API	실시간 검색	오늘 서울 날씨, 최근 뉴스 검색
Wikipedia API	백과사전 기반 정보 검색	개념 설명, 인물/사건 요약
Vector Store	벡터 검색 기반 문서 질의응답	내부 문서에서 유사 문장 검색하여 응답 생성





Agent의 활용 구조

복잡한 요청일 경우, 여러 도구를 순차적으로 사용 가능



사용자 입력

사용자 입력을
LLM이 분석

해당 목적에 맞는
Tool 선택

선택한 Tool 실행
(API 호출, 계산 등)

Tool의 결과를 바탕으로
LLM이 자연어 응답
생성

생성한 응답 출력

6. Memory

대화의 흐름을 기억하는 기능

- 기본적인 LLM은 한 번 질문하고 나면,
이전 대화를 전혀 기억하지 못함
- **Memory** 기능은 사용자와의 대화를 기억해서,
더 자연스럽게 연결된 대화 가능
 - 사용자의 이전 질문
 - LLM이 어떤 대답을 했는지
 - 어떤 주제에서 대화 중인지 (맥락)

```
from langchain_core.chat_history import InMemoryChatMessageHistory

# 세션 ID를 기준으로 메모리 생성
session_id = "history-session"
history = InMemoryChatMessageHistory()

# 사용자 메시지 추가
history.add_user_message("서울 날씨 어때?")

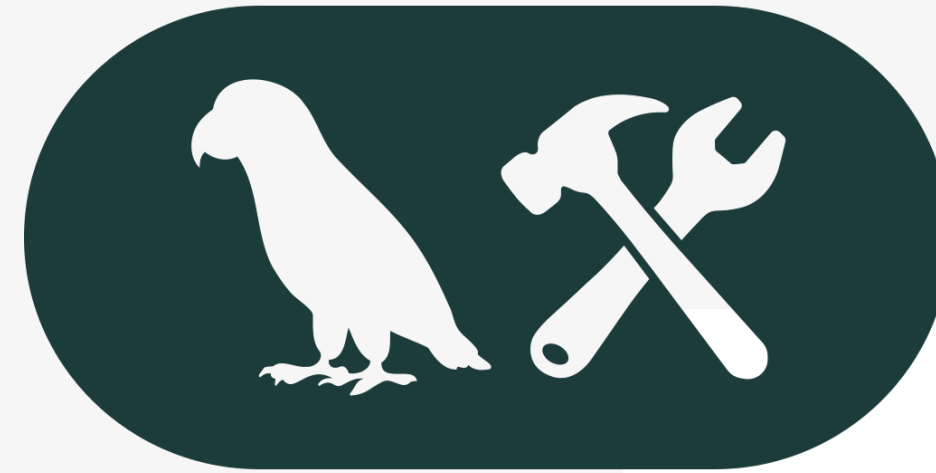
# AI 응답 추가
history.add_ai_message("오늘은 맑습니다.")

# history 예시
# [HumanMessage(content='서울 날씨 어때?',...),
#  AIMessage(content='오늘은 맑습니다.',...)]
```

7. Callback

실행 과정 추적 도구

- LLM이 어떻게 동작했는지, 어떤 도구를 사용했는지, 응답이 얼마나 걸렸는지 같은 **정보를 자동으로 기록**
 - 디버깅 : 문제가 생겼을 때 어디서 문제가 났는지 파악 가능
 - 모니터링 : 어떤 작업이 자주 쓰이는지 확인
 - 실험 기록 : 파라미터 변경 전후 비교 가능



도구 이름	설명
LangSmith	LangChain 공식 실험/추적 도구 (사용량 분석 등)
Weights & Biases	머신러닝 실험 기록용 도구와 연동 가능



Callback 툴 사용 예시

LangSmith

The screenshot displays the LangSmith web interface for a project named 'test_project'. The left sidebar shows navigation options like 'Runs', 'Threads', 'Monitor', and 'Setup'. The main area shows a list of runs, with 'pipeline' selected. The right panel provides details for the selected run, including the input prompt, the AI's response, and various performance metrics.

Run	Feedback	Metadata	Run ID	Trace ID
Input HUMAN I like Claude more, than GPT!				
Rendered Output AI That's great to hear! Claude is a fictional character known for being cunning and intelligent, while GPT is an AI language model known for generating text based on prompts. It's always fun to have a favorite character or personality in your life.				
START TIME				
END TIME				
TIME TO FIRST TOKEN N/A				
STATUS Success				
TOTAL TOKENS 65 tokens / \$0.0000815				
LATENCY 1.06s				
TYPE LLM				



Callback 툴 사용 예시

Weights & Biases

The screenshot displays the Weights & Biases (Wandb) interface. The top navigation bar shows the project path: **Projects** > **jurassic-park** > **Traces**. The left sidebar contains navigation options: Overview, Traces, Evals, Playground, Prompts, Models, Datasets, Scorers, and More. The main area is divided into two panels. The left panel, titled **Traces**, shows a list of operations with a filter bar and a table of traces. The right panel, titled **extract_dinos**, shows the details of a specific trace, including the **Call** tab, **Inputs**, and **Output**.

Traces Table:

Trace	Cost	Time	Status
extract_dinos	\$0.0014	1.85s	✓
openai.chat.completions.create	\$0.0014	1.84s	✓

Inputs:

Path	Value
sentence	I watched as a Tyrannosaurus rex (T. rex) chased after a Tric...

Output:

```
{
  "dinosaurs": [
    {
      "name": "Tyrannosaurus rex",
      "common_name": "T. rex",
      "diet": "carnivore"
    },
    {
      "name": "Triceratops",
      "common_name": "Trike",
      "diet": "herbivore"
    }
  ]
}
```